

PRG – TP1 – Algorithmes et Codages

Exercice 1 – “Suite des fourmis”

On veut afficher les 10 premiers termes de la suite définie par :

$u_0 = 1$	interprété comme	(1 un)
$u_1 = 1\ 1$	interprété comme	(2 un)
$u_2 = 2\ 1$	interprété comme	(1 deux, 1 un)
$u_3 = 1\ 2\ 1\ 1$	interprété comme	(1 un, 1 deux, 2 un)
$u_4 = 1\ 1\ 1\ 2\ 2\ 1$	interprété comme	(3 un, 2 deux, 1 un)
$u_5 = 3\ 1\ 2\ 2\ 1\ 1$	interprété comme	(1 trois, 1 un, 2 deux, 2 un)

Le premier terme u_0 étant donné, il s’agit de calculer u_i à l’aide de u_{i-1} pour $i > 0$.

```
1 package ci.miage.prg1.tpl;  
2  
3 public class Fourmis {  
4  
5     /**  
6      * @param s  
7      *          un terme de la suite des fourmis  
8      * @pre      s.length() > 0  
9      * @return    le terme suivant de la suite des fourmis  
10     */  
11     public static String next(String ui) { ... }  
12  
13     ...  
14  
15 }
```

Exercice 2 – Classement par insertion

On considère une suite d’entiers positifs lue au clavier et terminée par -1. Cette suite peut comporter plusieurs fois les mêmes valeurs. On veut construire le tableau d’entiers destiné à contenir les entiers distincts triés selon l’ordre croissant.

```
1 package ci.miage.prg1.tpl;
2
3 public class InsertionInteger {
4
5     private static final int SIZE_MAX = 10;
6     /**
7      * nombre d'entiers présents dans t, 0 <= size <= SIZE_MAX
8      */
9     private int size;
10    private int[] array = new int[SIZE_MAX];
11
12    public InsertionInteger() { ... }
13
14    /**
15     * @return copie de la partie remplie du tableau
16     */
17    public int[] toArray() { ... }
18
19    /**
20     * Si x n'appartient pas à array[0..size-1] et size < SIZE_MAX,
21     * size est incrémenté de 1, x est inséré dans array et les
22     * entiers array[0..size] sont triés par ordre croissant.
23     * Sinon array est inchangé.
24     *
25     * Exemple :
26     * pour x = 5, size = 3, array[0] = 1, array[1] = 6, array[2] = 8
27     * insertion délivre true, size = 4,
28     * array[0] = 1, array[1] = 5, array[2] = 6, array[3] = 8
29     *
30     * @param value
31     *         valeur à insérer
32     * @pre
33     *       les valeurs de array[0..size-1] sont triées par
34     *       ordre croissant
35     * @return false si x appartient à array[0..size-1] ou si
36     *         array est complètement rempli;
37     *         true si x n'appartient pas à array[0..size-1]
38     */
39    public boolean insert(int value) { ... }
40
41    /**
42     * array est rempli, par ordre croissant, en utilisant la fonction
43     * insert avec les valeurs lues par scanner.
44     *
45     * @param scanner
```

```

45     */
46     public void createArray(Scanner scanner)
47
48     @Override
49     public String toString() {...}
50 }

```

Exemple

en entrée $\implies$ en sortie
 3 18 1 4 3 2 1 3 -1 1 2 3 4 18

Écrire la classe `InsertionInteger`, ainsi qu'une classe de test qui permet de construire puis d'afficher, par ordre croissant, les entiers distincts lus au clavier.

Exercice 3 – Classement des doublets par insertion

On veut résoudre le même problème avec une suite de doublets d'entiers terminée par -1 (en position paire ou impaire). L'ordre et l'égalité sur les doublets sont définis respectivement par :

$(x1, y1)$ **less** $(x2, y2) \iff (x1 < x2) \text{ ou } (x1 == x2 \text{ et } y1 < y2)$
 $(x1, y1)$ **equals** $(x2, y2) \iff (x1 == x2) \text{ et } (y1 == y2)$

```

1 public class InsertionPair {
2     private static final int SIZE_MAX = 10;
3     private int size;
4     private Pair[] array = new Pair[SIZE_MAX];

```

Exemple

en entrée $\implies$ en sortie
 3 8 1 3
 1 4 1 4
 3 8 3 8
 1 3
 -1

À l'aide d'une classe `Pair` (décrivant les doublets *non modifiables*) et en apportant de légères modifications au programme de l'exercice 2, écrire la classe `InsertionPair`, ainsi qu'une classe qui permet de construire puis d'afficher, par ordre croissant, les doublets distincts positifs lus au clavier, ainsi que les doublets distincts lus dans un fichier texte dont le nom est une chaîne fournie au clavier.